

## HACK ATLANTIC 2026

# ScamGraph

## Project Proposal & Implementation Plan

*ScamGraph doesn't just tell you that something might be a scam. It shows you why.*

A two-person roadmap from suspicious content to an explainable risk score and interactive evidence graph.

### IN THIS DOCUMENT

Project Overview & Problem

Proposed Solution

Core Features & Optional Features

Technology Stack

Implementation Plan: Phases 0–9

Two-Person Responsibility Split

Definition of Done

## Project Overview

**ScamGraph** is a cybersecurity web application designed to help users identify and understand suspicious emails, text messages, and URLs.

Instead of simply labeling content as "safe" or "unsafe," ScamGraph analyzes the different components of a suspicious message and presents the results as an **interactive relationship graph**.

For example, a suspicious email may contain a sender address, domain, URL, claimed organization, and hosting infrastructure. ScamGraph extracts these components, checks them against threat-intelligence sources, performs additional heuristic analysis, and visually shows the relationships between them.

The goal is to make cybersecurity analysis understandable to everyday users while still providing useful technical information.

## Problem

Phishing and online scams frequently rely on impersonation, misleading domain names, malicious links, and social-engineering techniques.

A typical user might receive:

*"Your university account has been suspended. Sign in immediately at [unb-secure-login.xyz](#)."*

While an experienced user may recognize the warning signs, many people do not know **why** the message is suspicious.

Existing security tools can also return technical results that are difficult for non-technical users to interpret.

ScamGraph attempts to bridge that gap by answering two questions:

**"Is this suspicious?"**

and more importantly:

**"Why is this suspicious?"**

## Proposed Solution

Users submit suspicious content through the ScamGraph web interface.

SCAMGRAPH

Paste suspicious content

Your account has been suspended. Verify at...

[ ANALYZE ]

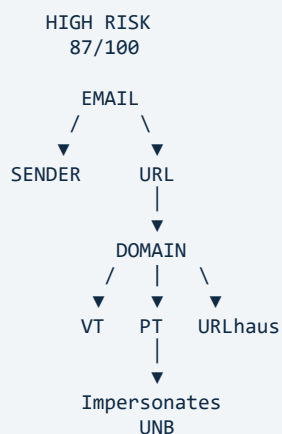
ScamGraph extracts indicators such as:

- URLs
- Domains
- Email addresses
- Claimed organizations
- Potential credential requests
- Suspicious language

These indicators are then analyzed using multiple sources.

```
INPUT -> Indicator Extraction
          |
          +-- VirusTotal / PhishTank / URLhaus
          |
          +-- Local Analysis -> Risk Engine
          |
          +-- Interactive Graph
```

The final result could look like:



Clicking a node provides additional evidence explaining its significance.

# Core Features

## 1. Suspicious Content Analysis

Users can paste an email, SMS message, or URL.

## 2. Indicator Extraction

The backend identifies URLs, domains, email addresses and other relevant information.

## 3. Threat Intelligence

ScamGraph queries external cybersecurity services such as:

- VirusTotal
- PhishTank
- URLhaus

These provide evidence about previously reported malicious URLs and domains.

## 4. Heuristic Detection

ScamGraph also performs its own analysis so that an unknown URL isn't automatically considered safe.

For example:

|                                     |     |
|-------------------------------------|-----|
| Domain resembles known organization | +15 |
| Credential request                  | +10 |
| Suspicious/urgent language          | +5  |
| Known phishing URL                  | +50 |
| Security vendor detections          | +30 |

The exact weights would be experimental rather than authoritative security ratings.

## 5. Explainable Risk Score

Results are summarized into a score such as:

|           |
|-----------|
| 87 / 100  |
| HIGH RISK |

Rather than presenting the number alone, ScamGraph shows which evidence contributed to it.

## 6. Interactive Scam Graph

This is the project's main visualization.

Relationships between indicators are represented as nodes and edges, allowing users to explore *why* ScamGraph considers something suspicious.

## Optional Features

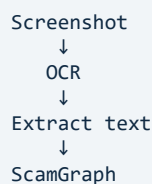
If the core application is completed early, we can add:

### AI Explanation

Convert technical results into plain language:

*"This message appears to impersonate UNB. The link uses a domain different from UNB's legitimate domain and attempts to direct the recipient to a login page."*

### Screenshot Analysis



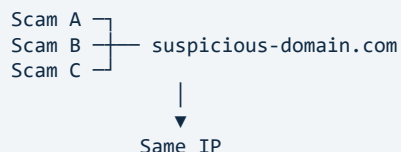
This would allow someone to screenshot a suspicious SMS or email and upload it directly.

### Investigation History

Save previous scans and their risk scores.

### Campaign Detection

Identify connections between separate scam reports:



This could eventually transform ScamGraph from an individual scam checker into a small threat-investigation platform.

# Technology Stack

For a two-person team, I'd keep it simple:

| Component           | Technology                     |
|---------------------|--------------------------------|
| Frontend            | React + Vite                   |
| Styling             | Tailwind CSS                   |
| Graph               | React Flow                     |
| Backend             | Python + FastAPI               |
| HTTP                | Axios / Fetch                  |
| Threat Intelligence | VirusTotal, PhishTank, URLhaus |
| Database            | SQLite initially               |
| AI                  | Optional LLM API               |
| Version Control     | GitHub                         |

I wouldn't introduce a complex database until there's actually a reason for one.

# Implementation Plan

## Phase 0 — Before Hackathon

### Both

Prepare your development environments without implementing ScamGraph itself unless the competition rules explicitly allow pre-written project code.

Have installed:

```
Git
Node.js
npm
Python
VS Code
Postman/Bruno (optional)
```

Create accounts/API keys required by your threat-intelligence sources.

Study:

- FastAPI basics
- React Flow basics
- VirusTotal API
- PhishTank API
- URLhaus API

And agree on the API structure you'll use between frontend and backend.

## Phase 1 — Project Setup

### Person A — Backend

Create:

```
backend/  
├── main.py  
├── analyzer/  
│   ├── extractor.py  
│   ├── scoring.py  
│   └── heuristics.py  
├── services/  
│   ├── virustotal.py  
│   ├── phishtank.py  
│   └── urlhaus.py  
└── requirements.txt
```

Main endpoint:

```
POST /analyze
```

Input:

```
{  
  "content": "Your account has been suspended..."  
}
```

### Person B — Frontend

Create:

```
frontend/  
├── src/  
│   ├── components/  
│   │   ├── InputPanel.jsx  
│   │   ├── RiskScore.jsx  
│   │   ├── ScamGraph.jsx  
│   │   └── EvidencePanel.jsx  
│   └── App.jsx  
└── package.json
```

Build the initial interface.

## Phase 2 — Indicator Extraction

### Person A

Turn:

```
Contact admin@fake-support.com  
  
Verify:  
https://unb-secure-login.xyz/login
```

into:

```
{  
  "emails": [  
    "admin@fake-support.com"  
  ],  
  "urls": [  
    "https://unb-secure-login.xyz/login"  
  ],  
  "domains": [  
    "fake-support.com",  
    "unb-secure-login.xyz"  
  ]  
}
```

Initially, regex + Python URL parsing is enough.

## Phase 3 — First Threat API

Don't implement three simultaneously.

Get **VirusTotal** working end-to-end first.

```
React  
↓  
FastAPI  
↓  
Extractor  
↓  
VirusTotal  
↓  
FastAPI  
↓  
React
```

Once this entire chain works, you've eliminated one of the biggest technical risks.

## Phase 4 — Risk Engine

Create an evidence system rather than scattering scoring logic throughout the application.

For example:

```
{
  "type": "domain_impersonation",
  "severity": "medium",
  "score": 15,
  "description": "Domain may impersonate UNB."
}
```

Multiple pieces of evidence become:

```
15 + 30 + 10 + 5
    ↓
   60
    ↓
HIGH RISK
```

That makes it very easy for the frontend to explain the score.

## Phase 5 — Build the Graph

### Person B

Convert backend results into React Flow nodes.

For example:

```
graph TD
    message --> contains
    contains --> URL
    URL --> domain
    domain --> unsecure[unb-secure-login.xyz]
    unsecure --> checked[checked by]
    checked --> VirusTotal
```

Clicking:

**unb-secure-login.xyz**

could open:

```
DOMAIN DETAILS

unb-secure-login.xyz

! Domain differs from claimed organization
! Suspicious login terminology
! VirusTotal detections

Risk contribution: +45
```

This is where I'd put a lot of the frontend effort.

## Phase 6 — Add Remaining Intelligence

Once VirusTotal works:

### Person A adds:

```
PhishTank
↓
URLhaus
↓
additional heuristics
```

But an unavailable API should **never break the analysis**.

For example:

|            |                |
|------------|----------------|
| VirusTotal | [OK] Available |
| PhishTank  | ! Unavailable  |
| URLhaus    | [OK] Available |

The application should simply continue with the evidence it has.

## Phase 7 — Integration Checkpoint

I'd establish a hard checkpoint Saturday evening.

By this point:

```
[OK] User can enter content
[OK] Backend receives it
[OK] Indicators extracted
[OK] At least one real threat API works
[OK] Risk score generated
[OK] Graph generated
[OK] Nodes can be inspected
```

If all seven checks pass:

### You have ScamGraph.

Everything else becomes optional.

## Phase 8 — Polish

Now both of you work together.

Add:

- Loading animations
- Error states
- Better graph positioning
- Risk-level indicators
- Evidence explanations
- Mobile/responsive layout
- Example input
- Clear/reset button

Most importantly, make the interface understandable **without you explaining it**.

## Phase 9 — Stretch Features

Only after the MVP is reliable.

My priority would be:

1. AI plain-English explanation  
↓
2. Screenshot/OCR  
↓
3. Investigation history  
↓
4. Snowflake integration  
↓
5. Campaign detection

Don't reach #4 while #1–3 are broken.

# Two-Person Responsibility Split

## Person A — Backend / Security

Own:

```
FastAPI
├── Indicator extraction
├── VirusTotal
├── PhishTank
├── URLhaus
├── Risk scoring
├── Heuristics
└── API responses
```

## Person B — Frontend / Visualization

Own:

```
React
├── Input UI
├── API integration
├── Risk display
├── React Flow
├── Evidence panel
├── Loading/error states
└── Visual polish
```

Once the two halves connect, both of you switch to testing and stretch features.

## Definition of Done

This is the sentence I'd keep at the top of your task board:

**A user can paste a suspicious message, click Analyze, and receive an explainable risk score with an interactive graph showing the relationships and evidence that contributed to the assessment.**

If you accomplish **that one sentence**, the project is complete.

Everything else is bonus.

And for your judging pitch, the central message becomes:

**ScamGraph doesn't just tell you that something might be a scam. It shows you why.**

That's the identity I'd build the entire project around.